1 Part 1

I have implemented my solution as a multi-modal classifier, chosen as a boosted tree model (LightGBM) to avoid overfitting on a relatively small dataset of ∼10,000 observations (OCR-scanned bounding boxes).

My repo consists of the following files,

- `data_extraction.py`: This scripts loads the JSON files and creates Pandas dataframes with each entry corresponding to a bounding box. The train and test set dataframes are stored as Parquet files.

- `feature_engineering.py`: Here we preprocess the data to create useful features for the model. There are four types of features:

  1. *Node height.* We observe that the "linking" entries define a *directed graph* in which the node heights $n_h = 0, 1, 2$ correspond to the classes Answer, Question and Header. (Presumably, the linking was obtained from the spatial relationships of the bounding boxes.) As the graph information is spread across multiple rows, we embed all nodes in a global graph (one per `page_id`) and compute the node height.

  2. *Layout information on bounding boxes.* Information on width, height, center coordinates, aspect ratio and area of the bounding boxes.

  3. *Word embedding vectors of the text in the bounding boxes.* We encode the text as word embedding vectors using the text embedding model BAAI/bge-base-en-v1.5 available from Hugging Face. We prepend the strings with the phrase "Represent this document bounding box text for layout classification: " to make use of the model's context-aware embedding.

  This step represents the words as 768-dimensional vectors. To avoid having the embedding vectors take up the majority of the features and have the boosted tree do all its splits on word embedding directions alone, we pipe the embedding vectors into a 32-dimensional PCA. (The dimension 32 is observed downstream to yield a better model than 16 or 24 dimensions.)

  4. *Ad-hoc punctuation features.* I expect words that contain "?" or ":" are Questions. Words of the form "02/07/2023", or words that contain currency symbols or a high fraction of numerical characters are likely to be Answers.

- `model.py`. We split the training dataset into a proper training training set use to train our classifier. We set aside a validation set for hyperparameter tuning (done with Optuna).

  The model has a classification accuracy of 87%. The confusion matrix and feature importance is shown below.

  Some preliminary error analysis is done at the bottom of the script.

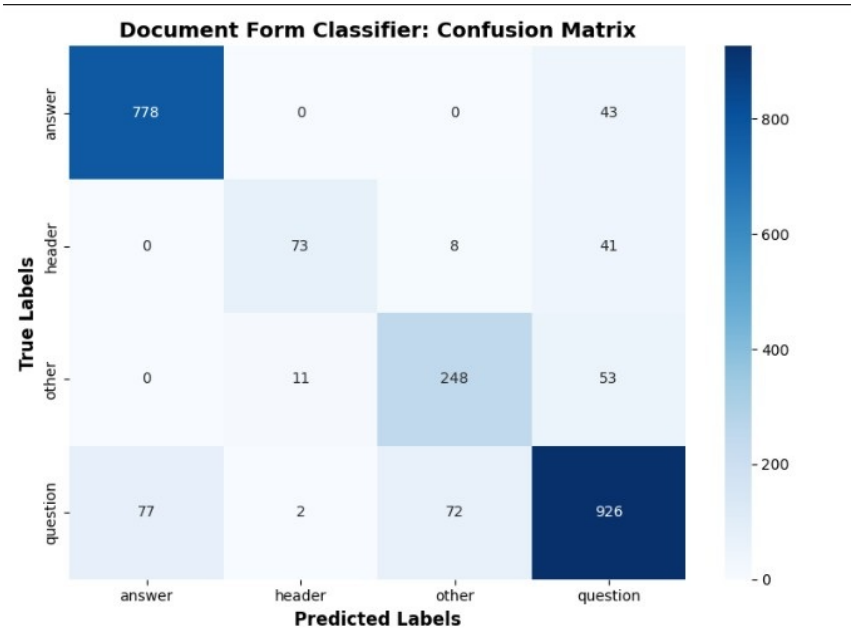

Figure 1: Confusion matrix

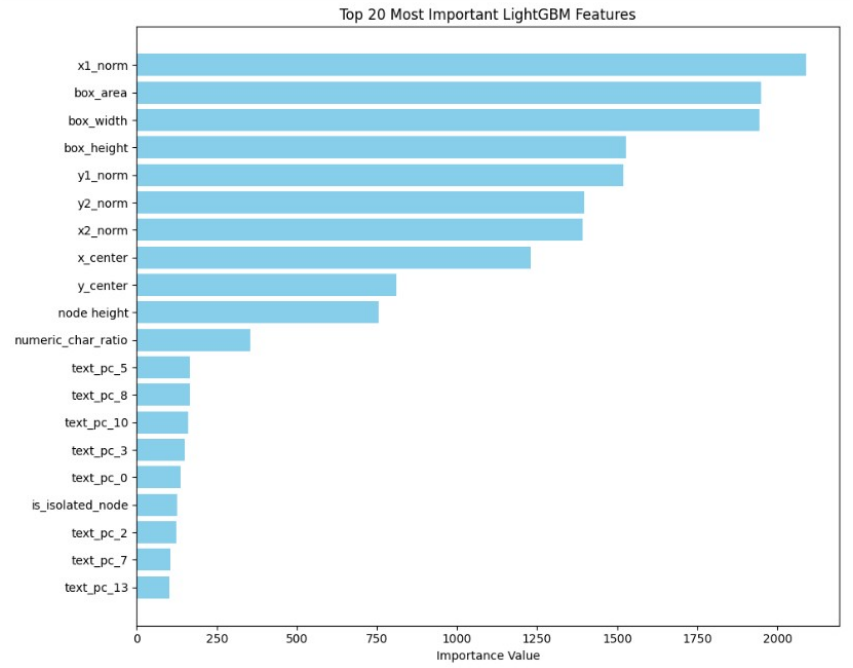

Figure 2: Feature importance, based on number many times a feature is used to split data across all trees.

2 Part 2: Paper Club

I would like to discuss `https://arxiv.org/pdf/2506.07972`, "HeuriGym: An Agentic Benchmark for LLM-Crafted Heuristics in Combinatorial Optimization" (ICLR 2026). The code is available on GitHub,
`https://github.com/cornell-zhang/heurigym`

The paper looks interesting because it offers a simple way for LLMs to uncover heuristic algorithms for combinatorial optimization. HeuriGym also allows automated improvement of existing heuristic algorithms rather than manual tuning, thereby potentially shortening development cycles considerably. The problems that could be addressed with this framework are vehicle routing, pickup and delivery with time windows etc.

With this application in mind, the paper can be viewed as describing Agentic LLMs that solve logistics problems.

HeuriGym can be used to generate solutions to the Pickup and Delivery with Time Windows problem as follows

1. Set up access to an LLM API (e.g., I used gemini-2.5-flash-lite). Then run `llm_solver_agent.py`. This will copy the following files from

   the `pickup_delivery_time_windows` subfolder: - `evaluator.py` - `feedback.py` - `main.py` - `run.py` - `solver.py` (ONLY FILE TO BE MODIFIED BY THE LLM) - `utils.py` - `verifier.py`

2. The new generated `solver.py` (generated from a prompt constructed from the input of the remaining files) can now be run on the data. This consists of 30 .pdptw (Pickup and Delivery Problem with Time Windows) files.

3. The results of the generated routes are stored in the repo folder `heurigym/visualization_outputs`

4. The costs produced by the human expert vs. LLM generated routes are stored in the repo folder `heurigym` as `human_expert_baseline.json` and `heurigym_cost_results.json` respectively.

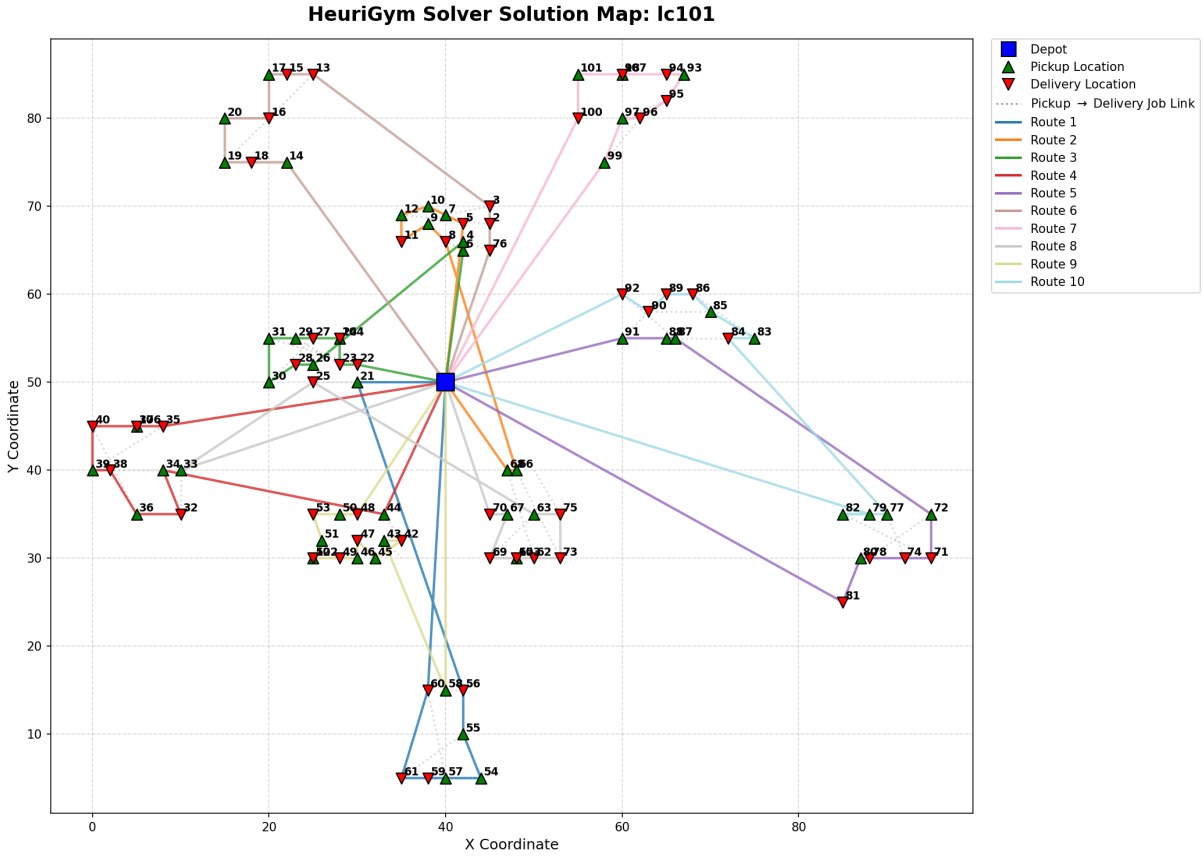

Figure 3: Example output of Pickup and Delivery routes generated with HeuriGym

3 Part 3: Research Case - Case 2

Given that OCR of the documents is available, I would go with Option 1.

Concretely, I would

1. Preprocess the OCR input with Recursive XY-Cut: separate the document into bounding boxes by projecting first onto the X-axis, then onto the Y-axis, splitting at the widest gap. Iterate until there are no gaps larger than a given threshold value.

2. Due to varying document resolution, scale all bounding box coordinates to a fixed integer scale between 0 and 1000. For example, letting (W, H) denote the width and height in pixels,

$$x_{\text{normalized}} = \left\lfloor \frac{x_{\text{OCR}}}{W} \times 1000 \right\rfloor , \tag{3.1}$$

$$y_{\text{normalized}} = \left\lfloor \frac{y_{\text{OCR}}}{H} \times 1000 \right\rfloor . \tag{3.2}$$

   We choose integers to limit token usage and make the input predictable for the LLM's vocabulary.

3. Interleave preprocessed bounding box data with OCR text in XML format (to limit formatting overhead, so as to save token).

   Our input may look like `<block bbox="120,450,140,580">Invoice Number:</block>`

   `<block bbox="120,590,140,710">INV-2026-89</block>`

4. Because the LLM is uni-modal, it doesn't understand spatial layout. To ensure the LLM doesn't hallucinate new coordinates for the bounding boxes, we demand that the LLM output a) must be JSON format, and b) the value of "bbox" in the JSON must be an array of 4 integer which match the coordinates inside the `<block bbox ...` in the input XML.

   In practice, we can do this by blocking invalid tokens at inference. We can implement this with a Pydantic schema.

Best Regards,
Kasper Jens Larsen